Verificación de Procedimientos para la Definición de Componentes Front-End

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

Este documento describe los componentes front-end de **MONK GRAIN**, la plataforma de compra y venta de café pergamino seco, con la justificación de cada uno.

La pregunta que responde no es «¿qué componentes tiene React?», sino **«¿cuándo algo merece ser un componente y cuándo no?»**. Convertir cada trozo de interfaz en una pieza propia produce un proyecto con doscientos archivos que nadie recuerda, y dejar todo en línea produce páginas de quinientas líneas donde la regla de negocio se esconde entre maquetación. Aquí queda escrito el criterio que se usó, y la medición en la que se apoya.

2. Objetivo

Describir los componentes front-end que se usan en el proyecto, justificando cada uno, y dejar establecido el procedimiento por el que se decide si algo debe serlo.

3. El procedimiento de decisión

Un trozo de interfaz se convierte en componente cuando cumple **al menos uno** de estos tres criterios. Si no cumple ninguno, se deja donde está:

Criterio	Qué significa	Ejemplo
1. Se repite	Aparece en dos o más pantallas. Copiarlo garantiza que un arreglo llegue a una copia y no a la otra	`ProductCard`, en el catálogo y en favoritos
2. Encierra una regla del dominio	Traduce un dato a una decisión. La traducción debe ser una sola en todo el producto	`YieldFactorReading`: «88,50» → «Excepcional»
3. Tiene estado propio	Gestiona algo que solo le concierne a él, y sacarlo descarga la página	`AddToCart`, con su cantidad y su mínimo

> **El criterio que no se usa: «es grande».** Un bloque largo de maquetación sin lógica ni repetición no gana nada al extraerse; solo añade un salto entre archivos para leer lo mismo.

4. La organización en tres capas

```
components/ui/         primitivas sin dominio       4 componentes
components/shared/     compartidos del producto      6 componentes
features/*/components  dominio, uno por módulo      14 componentes
hooks/                 lógica reutilizable           1 hook
```

Capa	Regla	Cómo se reconoce
`ui/`	No sabe nada de café. Serviría igual en otro proyecto	Si menciona «lote», «factor» o «pergamino», está en la capa equivocada
`shared/`	Es de este producto pero no de un módulo	La barra de navegación es de MONK GRAIN, pero no del catálogo ni del carrito
`features/`	Pertenece a un módulo y **espeja el backend**	`features/trading/` ↔ `modules/trading/` del servidor

El espejo con el backend no es simetría por gusto: cuando una regla cambia en el servidor, se sabe sin buscar qué carpeta del frontend hay que revisar.

5. Los componentes, con su justificación

5.1 Primitivas — `components/ui/`

Componente	Qué resuelve	Por qué existe
`Button`	Botón con variantes y tamaños	Sin él, cada pantalla inventa su botón y la interfaz deja de parecer una sola
`Badge`	Etiqueta de estado	Los estados se repiten en pedidos, posturas, devoluciones y liquidaciones
`Card`	Contenedor con su relleno	Unifica el ritmo visual de todos los listados
`Input`	Campo con sus estados de foco y error	El foco visible es un requisito de accesibilidad, no un detalle estético

5.2 Compartidos — `components/shared/`

Componente	Qué resuelve	Por qué existe
`Navbar` · `Footer`	Navegación global	Salen en las 43 páginas. Un enlace equivocado aquí son 43 enlaces rotos — ya ocurrió, y hoy una prueba lo vigila
`Logo`	La marca	Un solo sitio donde cambiarla
`CoffeeImage`	Foto del lote con su respaldo	Un lote sin foto no debe dejar un hueco roto; esto pone un marcador de la paleta
`StatCard`	Cifra con su rótulo y su énfasis	Los tres paneles —comprador, vendedor, administración— muestran cifras igual
`states`	`EmptyState`, `ErrorState`, `Skeleton`	**Los tres estados que toda pantalla con datos remotos tiene**, y que sin un componente se olvidan: lo más común es dibujar solo el caso con datos

5.3 De dominio — `features/*/components/`

Módulo	Componente	Por qué existe
`catalog`	`ProductCard`	Se repite en catálogo, favoritos y perfil del vendedor
	`CatalogFilters`	Encierra que el factor y la humedad se filtran por su **máximo**: en esos datos, menos es mejor
	`AddToCart`	Tiene estado propio y hace cumplir el mínimo de compra, que es regla del negocio
`producer`	`ProductForm`	La ficha de análisis del pergamino: factor, humedad, pasilla, broca, malla
	`VariantManager`	Las ofertas del lote, cada una con su mínimo y su precio por kilo
	`ImageManager`	Subida con orden y portada
`trading`	`CoverageBar`	**Una solicitud grande no la cubre un solo caficultor.** Comprador y vendedor necesitan ver cuánto falta
	`YieldFactorReading`	Traduce el factor a su lectura comercial. Dos traducciones del mismo número en pantallas distintas sería peor que ninguna
	`OfferResponse`	Muestra el motivo del rechazo **completo**: es lo que le dice al caficultor qué ajustar
	`DemandCard`	La misma tarjeta para el comprador y el vendedor, con `href` distinto
`reviews`	`Stars` · `ProductReviews`	La calificación aparece en la ficha, en el perfil y en la moderación
`account`	`AddressForm`	Departamento y municipio en cascada contra el catálogo DANE
`returns`	`ReturnReceipt`	La constancia con número de radicación, que la ley exige

5.4 Lógica reutilizable — `hooks/`

`useProtectedResource` no dibuja nada: encapsula **comportamiento**. Es la capacidad que los hooks añadieron a React y que las clases no tenían.

Nace de una duplicación medida en el propio repositorio:

```
$ grep -rl "session.getAccessToken" web/src/app | wc -l
22        páginas leen el token a mano

$ grep -rl "requestError.status === 403" web/src/app | wc -l
14        repiten el mismo manejo del prohibido

$ grep -rl "const [isLoading, setIsLoading] = useState(true)" web/src/app | wc -l
28        repiten el trío cargando / error / prohibido
```

Veinticinco líneas copiadas veintiocho veces. Y lo que convierte eso en un defecto y no en una incomodidad: **basta con que una copia olvide el `finally`** para que esa pantalla se quede cargando para siempre, y nadie lo nota hasta que alguien lo reporta.

6. Reglas de escritura de un componente

Regla	Motivo
Nombre en `PascalCase`, archivo igual al componente	Encontrarlo por el nombre que se ve en el JSX
Propiedades tipadas, sin `any`	`tsc` en modo estricto rechaza el envío
Un componente no modifica lo que recibe	Avisa con una función. Así, cuando algo se ve mal, se busca hacia arriba
Las consultas de las pruebas van por texto accesible	Un cambio de maquetación no debe romper una prueba de una regla
El comentario explica el porqué	El código ya dice qué hace; lo que no puede decir es qué se descartó

6.1 Un componente como ejemplo

`CoverageBar` reúne las cinco reglas y una decisión de dominio:

```
export function CoverageBar({
  wantedKg, coveredKg, remainingKg,
}: {
  wantedKg: number; coveredKg: number; remainingKg: number;
}) {
  const percentage = wantedKg > 0 ? Math.round((coveredKg / wantedKg) * 100) : 0;

  return (
    <div>
      <span>{formatKilosWithCargas(coveredKg)} de {formatKilosWithCargas(wantedKg)}</span>
      <span>{percentage} %</span>

      {/* role y aria: un lector de pantalla no ve el ancho de un div */}
      <div role="progressbar" aria-valuenow={coveredKg} aria-valuemax={wantedKg}
           aria-label={`Cubierto ${percentage} por ciento`}>
        <div style={{ width: `${percentage}%` }} />
      </div>
    </div>
  );
}
```

Muestra los kilos y el porcentaje juntos a propósito. El porcentaje solo engaña —«80 %» de una solicitud pequeña puede ser menos café que el 20 % de una grande— y los kilos solos obligan a dividir mentalmente. Esa decisión está fijada por una prueba.

7. Verificación

El procedimiento se comprueba, no se declara:

Qué se verifica	Cómo	Estado
Tipos correctos en todos los componentes	`tsc --noEmit` en modo estricto	Sin hallazgos
Reglas de React y de los hooks	`eslint`	Sin hallazgos
Formato uniforme	`prettier --check`	Sin hallazgos
Comportamiento de los componentes	`vitest` con Testing Library	**53 pruebas**
Que la navegación no prometa páginas inexistentes	Prueba que cruza los `href` contra el árbol de rutas	3 pruebas

8. Conclusión

El proyecto tiene 24 componentes y un hook, repartidos en tres capas con una regla clara de pertenencia: las primitivas no saben de café, los compartidos son del producto pero no de un módulo, y los de dominio espejan los módulos del servidor.

Lo que distingue este inventario de una lista es el criterio: un trozo de interfaz se extrae porque **se repite**, porque **encierra una regla** o porque **tiene estado propio**. Nunca por ser grande. La evidencia siguiente aplica ese criterio al módulo de oferta y demanda.